

Tuần 5 — Messaging (SQS/SNS/Kinesis) + Step Functions + ElastiCache + RDS Proxy → HẾT Domain 1

Domain: Domain 1 – Development (32%) · **Thời lượng:** ~11h (4 buổi) · **Vị trí:** Tuần 5/5 — KẾT THÚC Domain 1, có CHECKPOINT

Điều hướng:  Tuần 4 ·  Kế hoạch tổng · Tuần 6 

Mục tiêu tuần này

- **Phân biệt được** SQS vs SNS vs Kinesis chỉ trong 5 giây khi đọc keyword đề (decouple / fan-out / stream + replay).
- **Tự tay** dựng fan-out SNS → 2 SQS và kiểm chứng cả 2 queue cùng nhận message.
- **Cấu hình được** SQS + Dead-Letter Queue với `maxReceiveCount`, giải thích khi nào message rơi vào DLQ.
- **Viết được** 1 state machine Step Functions (ASL) có Choice + Retry + Catch.
- **Giải thích được** caching strategy: lazy loading (cache-aside) vs write-through + vai trò TTL; chọn Redis hay Memcached.
- **Chốt Domain 1:** đạt $\geq 70\%$ ở MINI-MOCK Domain 1 (~25 câu) trước khi sang Tuần 6.

Nội dung học chi tiết

Buổi A — Lý thuyết (~3h)

1. SQS — hàng đợi decouple (rất hay hỏi)

- Standard vs FIFO:

- **Standard**: at-least-once (có thể trùng), thứ tự **best-effort**, throughput gần như **không giới hạn**. (Mới: hỗ trợ **Fair Queues** dùng **MessageGroupId** để giảm noisy-neighbor trong multi-tenant).
- **FIFO**: **exactly-once processing**, giữ đúng thứ tự; bắt buộc **MessageGroupId** (phân luồng thứ tự) + **MessageDeduplicationId** (chống trùng); dedup window **5 phút**. Throughput cơ bản: **300 msg/s** (tới **3.000 msg/s** khi batching 10 msg). Bật **high throughput mode** (scale bằng nhiều message group) → hạn mức nâng cao vượt bậc theo region:
 - **Region chính** (**us-east-1**, **us-west-2**, **eu-west-1**): lên tới **70.000 TPS** (không batch) / **700.000 msg/s** (có batch).
 - **Region phụ** (**us-east-2**, **eu-central-1**): tới **19.000 TPS** / **190.000 msg/s**.
 - **Region APAC** (Singapore, Tokyo, Sydney...): tới **9.000 TPS** / **90.000 msg/s**.
 - (Con số **~30.000 msg/s** hay gặp trong đề thi là mức trần cũ trước đây).
- **Visibility timeout**: message đang xử lý bị "ẩn" khỏi consumer khác. Mặc định **30 giây**, tối đa **12 giờ**. Xử lý lâu hơn timeout → message tái xuất hiện → **xử lý trùng** (dùng **ChangeMessageVisibility** để gia hạn).
- **Message retention**: mặc định **4 ngày**, cấu hình từ **~60 giây** → **14 ngày**.
- **Long vs short polling**: long polling **WaitTimeSeconds** tối đa **20 giây** → giảm empty response & **chi phí**; short polling trả về ngay (nhiều request rỗng hơn).
- **DLQ + maxReceiveCount & DLQ Redrive**: message bị receive quá **maxReceiveCount** lần mà chưa xóa → đẩy sang **Dead-Letter Queue** để điều tra (poison message). Hỗ trợ **DLQ Redrive** (Console hoặc API **StartMessageMoveTask**) để đẩy lại message từ DLQ về queue nguồn sau khi fix lỗi code mà không cần viết script riêng.
- **Delay queue**: trì hoãn giao message tối đa **15 phút**.
- **Message lớn**: **SQS** message tối đa hiện tại là **1 MiB (1.048.576 bytes)**, cấu hình từ 1 KiB → 1 MiB (*trước đây & trong nhiều câu hỏi đề thi cũ thường lấy mốc kinh điển 256 KB*); payload vượt quá hạn mức → dùng thư viện **SQS Extended Client Library** (lưu payload ở **S3**, gửi con trỏ trong queue), hỗ trợ tới **2 GB**. (Lưu ý: chỉ dùng qua **SDK Extended Client Library**, không làm trực tiếp qua **AWS CLI** / **Console**).

2. SNS — pub/sub, fan-out

- Mô hình **publish/subscribe**: 1 message → **nhiều** subscriber. Subscriber: **SQS**, **Lambda**, **HTTP(S)**, email, SMS, mobile push, Firehose, EventBridge.

- **Fan-out SNS → nhiều SQS**: publish 1 lần, mỗi queue nhận 1 bản → xử lý độc lập (decouple + broadcast). Mẫu kiến trúc kinh điển của đề.
- **Message filtering**: gắn **filter policy** (JSON) trên subscription → mỗi subscriber chỉ nhận message khớp điều kiện → khỏi tự lọc trong code. Hỗ trợ 2 scope:
 - **MessageAttributes** (mặc định): lọc theo metadata thuộc tính.
 - **MessageBody (payload-based filtering)**: lọc trực tiếp theo cấu trúc JSON bên trong payload message mà không cần sửa code publisher.
- **FIFO topic**: giữ thứ tự & khử trùng (thường ghép với **SQS FIFO**). Message tối đa **256 KB** (*Lưu ý: SNS vẫn giữ giới hạn cứng 256 KB, nên luồng fan-out SNS → SQS sẽ bị giới hạn bởi SNS*).

3. Kinesis Data Streams vs Amazon Data Firehose (phân biệt sống còn)

- **Kinesis Data Streams (KDS)**: real-time streaming. Đơn vị scale = **shard**: mỗi shard ghi **1 MB/s HOẶC 1000 records/s**, đọc **2 MB/s**; record $\leq 1 \text{ MB}$. Ordered **theo partition key**. Retention mặc định **24 giờ**, tối đa **365 ngày** → **replay** được. Hỗ trợ **nhiều consumer**; **enhanced fan-out** cho mỗi consumer **2 MB/s riêng mỗi shard**. Ghi quá hạn mức → **ProvisionedThroughputExceededException** (throttle):
 - **Cách xử lý throttle**: Exponential backoff & retry trong code producer; chọn Partition Key phân bổ đều (tránh hot shard); **Resharding** (split shard để tăng throughput, merge shard để giảm chi phí).
 - **Capacity mode**: **on-demand** (AWS tự quản shard, không cần capacity planning, trả theo throughput) vs **provisioned** (tự đặt số shard).
- **Amazon Data Firehose** (trước đây là **Kinesis Data Firehose**): **near-real-time, KHÔNG replay**. Tự **nạp** dữ liệu vào **S3 / Redshift / OpenSearch / Splunk / Snowflake / Apache Iceberg**; có **buffering** theo size (1–128 MB) hoặc time (60–900s). Không quản shard, không code consumer. Tích hợp **Lambda** để biến đổi dữ liệu in-flight (chuyển đổi định dạng JSON sang Parquet/ORC qua AWS Glue, giải nén, format).

4. Step Functions — điều phối workflow

- Định nghĩa bằng **ASL** (Amazon States Language, JSON). Các state: **Task, Choice, Parallel, Map, Wait, Pass, Succeed, Fail**.
- **Xử lý lỗi ngay trong workflow**: **Retry** (thử lại có backoff) + **Catch** (bắt lỗi, rẽ nhánh dự phòng) → không cần nhét retry vào code Lambda.
- **Tính năng mở rộng**: **Distributed Map** (chạy song song tới 10.000 child execution, tối ưu duyệt hàng triệu file S3), **TestState API** (kiểm thử từng state độc

lập), **HTTPS Tasks** (gọi REST API ngoài không cần Lambda).

- **Standard vs Express:**

Tiêu chí	Standard	Express
Thời gian chạy tối đa	tới 1 năm	tới 5 phút (gọi <i>Sync</i> từ Console <i>timeout 60s</i> , muốn đủ 5p dùng <i>SDK/CLI</i>)
Ngữ nghĩa	exactly-once	Asynchronous: at-least-once · Synchronous (StartSyncExecution): at-most-once
Lịch sử thực thi	Lưu tới 90 ngày trong Step Functions (visual audit)	KHÔNG lưu trong Step Functions; bắt buộc gửi ra CloudWatch Logs
Service Integrations	Đủ 3 pattern (Request-Response, .sync , .waitForTaskToken)	Chỉ Request-Response; KHÔNG hỗ trợ .sync và .waitForTaskToken
Tính tiền	theo state transition	theo số lần chạy + thời lượng + bộ nhớ
Hợp cho	workflow dài, kiểm toán, bước con người	high-volume , event ngắn, IoT, API sync

B Buổi B — Hands-on (~3.5h)

 Lab cầm tay chỉ việc (từng bước + lệnh + code): labs.md.

Lab 1 — Fan-out **SNS** → 2 **SQS**

1. Tạo 2 queue: **orders-analytics** và **orders-billing**.

```
aws sqs create-queue --queue-name orders-analytics
aws sqs create-queue --queue-name orders-billing
```

2. Tạo topic:

```
aws sns create-topic --name new-orders
```

3. Subscribe từng queue vào topic (`--protocol sqs, --notification-endpoint` = ARN queue). Bật **raw message delivery** để **SQS** nhận đúng body.
4. Gắn **access policy** cho mỗi queue cho phép **SNS SendMessage** (điều kiện `aws:SourceArn` = ARN topic) — nếu thiếu, publish thành công nhưng queue **không nhận** được.
5. Publish 1 message:

```
aws sns publish --topic-arn <topic-arn> --message '{"orderId":"1001"}'
```

6. **Kiểm chứng:** `receive-message` ở **cả hai** queue → cả hai đều có bản sao. Đó là fan-out.

Lab 2 — SQS + DLQ + `maxReceiveCount`

1. Tạo queue `payments-dlq` (dead-letter) và queue chính `payments`.
2. Gán **RedrivePolicy** cho `payments`: `deadLetterTargetArn` = ARN của `payments-dlq`, `maxReceiveCount` = **3**.
3. Gửi 1 message vào `payments`. Lặp lại: `receive-message` nhưng **KHÔNG delete-message** (giả lập xử lý thất bại) → sau khi hết visibility timeout message lại xuất hiện.
4. Sau lần receive thứ **4** (vượt `maxReceiveCount` = 3) → message tự chuyển sang `payments-dlq`. Xác nhận bằng `receive-message` trên DLQ.

Lab 3 — State machine **Step Functions** có **Choice** + **Retry** + **Catch**

1. Tạo state machine loại **Standard** với ASL rút gọn:

```
{
  "Comment": "Order flow",
  "StartAt": "CheckAmount",
  "States": {
    "CheckAmount": {
      "Type": "Choice",
      "Choices": [
        { "Variable": "$.amount", "NumericGreaterThan": 1000, "Next":
"ManualReview" }
      ],
      "Default": "ChargeCard"
    },
    "ChargeCard": {
      "Type": "Task",
      "Resource": "<lambda-arn>",
      "Retry": [
        { "ErrorEquals": ["States.TaskFailed"], "IntervalSeconds": 2,
"MaxAttempts": 3, "BackoffRate": 2.0 }
      ]
    }
  }
}
```

```

    ],
    "Catch": [
      { "ErrorEquals": ["States.ALL"], "Next": "ChargeFailed" }
    ],
    "End": true
  },
  "ManualReview": { "Type": "Succeed" },
  "ChargeFailed": { "Type": "Fail", "Error": "ChargeError" }
}

```

2. Chạy execution với input `{"amount": 500}` → đi nhánh **ChargeCard**; với `{"amount": 2000}` → đi **ManualReview**.
3. Cho **Lambda** fail để quan sát **Retry** (3 lần, backoff 2x) rồi rơi vào **Catch** → **ChargeFailed**. Xem sơ đồ execution trong console.

Lab 4 — Bảng so sánh tự viết

- **Tự tay** viết lại bảng "SQS vs SNS vs Kinesis" dưới đây bằng trí nhớ, rồi đối chiếu. Đây là bảng bị hỏi nhiều nhất của Domain 1.

C Buổi C — Bổ sung (~2.5h)

KHI NÀO dùng **SQS** / **SNS** / **Kinesis** — bảng quyết định:

Tiêu chí	SQS	SNS	Kinesis Data Streams
Mô hình	Queue (1 producer → 1 nhóm consumer cùng đọc, mỗi msg xử lý 1 lần)	Pub/Sub push tới nhiều subscriber	Stream real-time, nhiều consumer đọc song song
Thứ tự	Standard: best-effort; FIFO : có	FIFO topic: có	Có (theo partition key trong shard)
Replay lại dữ liệu cũ	✗ (xóa sau khi xử lý / hết retention)	✗	✓ (đọc lại trong retention 24h–365 ngày)
Nhiều consumer nhận cùng dữ liệu	✗ (dùng fan-out SNS → SQS)	✓	✓ (nhiều app đọc cùng stream)

Tiêu chí	SQS	SNS	Kinesis Data Streams
Dùng khi	decouple đơn giản, xử lý theo job	broadcast / fan-out 1 → N	analytics / ordered / replay / high-throughput

ElastiCache — caching layer

- Engine hiện tại: **Valkey / Redis OSS / Memcached. ElastiCache Serverless** (2023): không quản node/capacity, tự scale, tạo < 1 phút, trả theo dung lượng+compute.
- So sánh với Amazon MemoryDB (Bây đề DVA-C02):**
 - ElastiCache**: là **tầng Caching**, dữ liệu có thể mất khi node crash (không đảm bảo durability 100%).
 - Amazon MemoryDB for Redis/Valkey**: là **Primary In-Memory Database**, độ bền cao nhờ **Multi-AZ Transactional Log** (đáp ứng bài toán cần đọc/ghi microsecond + bền vững tuyệt đối).
- Redis vs Memcached:**

	Redis (hoặc Valkey)	Memcached
Persistence	✅ (snapshot RDB / AOF)	❌
Replication + HA/failover	✅ (Multi-AZ, Read Replicas)	❌
Kiểu dữ liệu	phong phú (sorted set, list, hash...), pub/sub	key-value đơn giản
Đa luồng (multi-thread)	❌ Command execution đơn luồng (đảm bảo atomic); I/O đa luồng	✅ Thuần kiến trúc đa luồng
Chọn khi	cần HA, cấu trúc dữ liệu, leaderboard, session bền	cache thuần túy, scale ngang CPU nhiều core

- Caching strategy:**
 - Lazy loading (cache-aside):** đọc cache trước; **miss** → query DB → ghi vào cache. Ưu: chỉ cache dữ liệu thật sự được dùng. Nhược: lần miss đầu chậm; dữ liệu có thể **stale**.
 - Write-through:** mỗi lần ghi DB thì **ghi luôn** vào cache → cache luôn mới. Nhược: ghi nhiều dữ liệu có thể không bao giờ được đọc (tốn bộ nhớ).

- **TTL**: đặt thời hạn key để **chống stale**; thường ghép với lazy loading để dữ liệu tự hết hạn.

RDS Proxy — gom connection cho Lambda

- **Lambda** scale → hàng nghìn connection đồng thời đập vào DB = "**connection storm**" → DB cạn kết nối.
- **RDS Proxy pool (gom) và tái dùng** connection, giảm áp lực mở/đóng liên tục; giảm thời gian failover lên tới 66%.
- Tích hợp **Secrets Manager** (lấy credential) và hỗ trợ **IAM auth**.
- **Điểm thi quan trọng về mạng & tính năng**:
 - **Vị trí mạng**: **RDS Proxy bắt buộc nằm trong VPC, KHÔNG thể truy cập Public**. Lambda muốn gọi proxy phải kết nối qua VPC.
 - **Reader Endpoints (Read-Only)**: Hỗ trợ tạo endpoint đọc riêng biệt cho Aurora / Multi-AZ DB clusters để scale traffic đọc.
 - **Session Pinning**: Các câu lệnh query > 16 KB, prepared statement, temporary tables khiến proxy pin cố định vào 1 connection → mất tác dụng pooling.

Đọc thêm: **SQS** FAQ (visibility timeout, FIFO, DLQ redrive), **SNS** message filtering (MessageBody), **Kinesis** Developer Guide (shard, enhanced fan-out), **Amazon Data Firehose** Developer Guide.

Buổi D — Practice + Review (~2h)

 **Bộ câu hỏi luyện tập của tuần**: [questions.md](#) — đáp án & giải thích: [answers.md](#). (bằng tiếng Anh — văn phong đề thật để làm quen đề.)

- Làm bộ câu hỏi chủ đề messaging + orchestration + caching.
- ★ **MINI-MOCK Domain 1 (~25 câu)** trộn toàn bộ Tuần 1 → 5. **Ghi số câu sai**, phân loại theo dịch vụ.
- **Spaced repetition**: ôn lại flashcard số liệu theo mốc **1 / 3 / 7 ngày** (số liệu **SQS**, **Kinesis** rất dễ quên).
- Chỉ sang Tuần 6 khi mini-mock **≥70%**.

 **PHẢI NHỚ tuần này**

Fact	Con số / Ghi nhớ
SQS message tối đa	1 MiB (1.048.576 bytes) mới nhất (đề thi cũ hay lấy mốc 256 KB); vượt hạn mức → SDK Extended Client + S3 , tới 2 GB (không hỗ trợ qua CLI/Console)
Visibility timeout	mặc định 30 giây , tối đa 12 giờ
Message retention	mặc định 4 ngày , cấu hình từ ~ 60 giây → 14 ngày
Long polling WaitTimeSeconds	tối đa 20 giây
Delay queue	tối đa 15 phút
SQS FIFO throughput	Cơ bản: 300 msg/s (tới 3.000 msg/s khi batching); Bật high throughput mode → lên tới 70.000 TPS / 700.000 msg/s (region chính), scale qua nhiều message group; dedup window 5 phút
SQS Standard	at-least-once, thứ tự best-effort, throughput ~không giới hạn; hỗ trợ Fair Queues qua MessageGroupId
SQS DLQ Redrive	Redrive trực tiếp từ DLQ về source queue qua Console hoặc API StartMessageMoveTask
SNS message	tối đa 256 KB (luồng fan-out SNS → SQS bị giới hạn bởi SNS); hỗ trợ filter policy (scope: MessageAttributes hoặc MessageBody) + FIFO topic
Kinesis shard (ghi)	1 MB/s HOẶC 1000 records/s ; đọc 2 MB/s ; record $\leq$ 1 MB
Kinesis retention	mặc định 24 giờ , tối đa 365 ngày → replay được
Kinesis enhanced fan-out	2 MB/s riêng mỗi consumer/shard qua HTTP/2 push
Kinesis throttle	ProvisionedThroughputExceededException (xử lý: backoff/retry, phân bổ partition key đều, resharding split/merge)
Kinesis capacity mode	on-demand (AWS tự quản shard, trả theo throughput) vs provisioned (tự quản số shard)
Amazon Data Firehose	Near-real-time, KHÔNG replay, tự load vào S3/Redshift/OpenSearch/Splunk/Snowflake/Iceberg; transform qua Lambda

Fact	Con số / Ghi nhớ
Step Functions Standard	tối 1 năm , exactly-once , tính theo state transition, lưu history 90 ngày, hỗ trợ .sync , .waitForTaskToken , Distributed Map
Step Functions Express	tối 5 phút , Async: at-least-once , Sync: at-most-once ; KHÔNG lưu history trong SFN (phải dùng CloudWatch Logs); chỉ hỗ trợ Request-Response
RDS Proxy	Connection pooling cho Lambda, bắt buộc trong VPC (không public) , hỗ trợ Reader endpoints, IAM & Secrets Manager auth

! Bẫy đề hay gặp

- Thấy "nhiều consumer cần **cùng** dữ liệu" → dễ chọn **SQS**, nhưng **SQS** mỗi message chỉ 1 consumer xử lý → đúng là **fan-out SNS → SQS** hoặc **Kinesis**.
- Thấy "cần **replay** / đọc lại dữ liệu cũ" → chọn nhầm **SQS/SNS** (không replay được) → đúng là **Kinesis Data Streams**.
- Thấy "tự động nạp stream vào **S3/Redshift**, không cần code" → chọn nhầm **KDS** → đúng là **Amazon Data Firehose** (KDS phải tự viết consumer).
- Thấy "message vượt hạn mức (đề thi thường lấy mốc > 256 KB hoặc > 1 MiB)" → tưởng phải đổi dịch vụ → đúng là **SQS Extended Client + S3** (tối 2 GB, bắt buộc dùng thư viện SDK Extended Client).
- Xử lý message **lâu hơn visibility timeout** → tưởng an toàn → thực ra message **tái xuất hiện** và bị xử lý trùng → gia hạn bằng **ChangeMessageVisibility**.
- Thấy "cần đúng **thứ tự** + không trùng" → chọn **SQS Standard** là **sai** → phải **FIFO** (kèm **MessageGroupId** + **MessageDeduplicationId**).
- Thấy "subscriber muốn lọc message dựa trên nội dung JSON payload không có attributes" → chọn nhầm nhiều topic → đúng là **SNS Filter Policy với scope MessageBody**.
- Thấy "cần xử lý lại các message lỗi trong DLQ sau khi sửa bug" → chọn nhầm viết script phức tạp → đúng là dùng tính năng **SQS DLQ Redrive (StartMessageMoveTask)**.
- Thấy "**Lambda** gây **connection storm** tới RDS" → tưởng phải tăng size DB → đúng là **RDS Proxy** gom connection (*chú ý: RDS Proxy nằm trong VPC, không bao giờ public*).

- Thấy "cần in-memory database có độ bền Multi-AZ transactional log (không phải chỉ là cache)" → chọn nhầm ElastiCache → đúng là **Amazon MemoryDB**.
- Thấy "troubleshoot execution history của Step Functions Express" → tìm trong SFN console là **sai** (SFN không lưu) → phải kiểm tra trong **Amazon CloudWatch Logs**.
- Thấy "retry/điều phối nhiều bước, bắt lỗi, chờ" nhét hết vào 1 **Lambda** → đúng ra dùng **Step Functions** (**Retry/Catch/Choice**).



Phản xạ nhanh (keyword → đáp án)

Thấy từ khoá	Bật ngay
decouple đơn giản, 1 nhóm consumer, job queue	SQS (Standard)
đúng thứ tự + không trùng	SQS FIFO (MessageGroupId + MessageDeduplicationId)
poison message / message lỗi lặp lại	DLQ + maxReceiveCount
đẩy lại message từ DLQ về queue chính sau khi fix bug	SQS DLQ Redrive (StartMessageMoveTask)
message SQS vượt quá hạn mức (hoặc để cho > 256 KB)	SQS Extended Client Library + S3 (tới 2 GB)
broadcast / fan-out 1 → N	SNS (hoặc SNS → nhiều SQS)
chỉ nhận message khớp thuộc tính	SNS filter policy (scope: MessageAttributes)
chỉ nhận message khớp nội dung JSON payload	SNS filter policy (scope: MessageBody)
real-time, ordered, nhiều consumer, replay	Kinesis Data Streams
"no shard/capacity management"	Kinesis on-demand
throughput riêng cho từng consumer qua HTTP/2 push	enhanced fan-out
tự nạp stream vào S3/Redshift/OpenSearch	Amazon Data Firehose (tên cũ Kinesis Firehose)

Thấy từ khoá	Bật ngay
điều phối workflow nhiều bước, Retry/Catch	Step Functions
workflow high-volume, ngắn (<5 phút), audit qua CW Logs	Step Functions Express
gọi sync workflow từ API Gateway trả kết quả ngay	Step Functions Synchronous Express (StartSyncExecution)
cache leaderboard / HA / pub-sub / session bền	ElastiCache for Redis / Valkey
cache đơn giản, đa luồng, scale ngang	ElastiCache for Memcached
in-memory database tốc độ cao + bền vững Multi-AZ log	Amazon MemoryDB (Primary DB, khác ElastiCache là cache)
Lambda mở quá nhiều connection tới RDS (trong VPC)	RDS Proxy



Lab checklist

- ☐ Dựng fan-out **SNS** → 2 **SQS**, xác nhận cả 2 queue nhận cùng message.
- ☐ Cấu hình access policy cho phép **SNS** gửi vào **SQS**.
- ☐ Tạo **SQS** + DLQ, đặt **maxReceiveCount**, đẩy được 1 message rơi vào DLQ.
- ☐ Viết state machine **Step Functions** có **Choice** + **Retry** + **Catch**, chạy 2 nhánh input.
- ☐ Tự viết lại bảng so sánh **SQS** / **SNS** / **Kinesis** bằng trí nhớ.

Cổng tự kiểm tra (phải trả lời trôi chảy mới sang tuần sau)

- Cần đúng thứ tự + nhiều consumer + replay dữ liệu cũ → chọn gì? Đáp án gọn: Kinesis Data Streams** (ordered theo partition key, nhiều consumer, retention tới 365 ngày cho phép replay).
- Decouple đơn giản, chỉ 1 consumer xử lý mỗi message → chọn gì? Đáp án gọn: SQS** (Standard nếu không cần thứ tự; FIFO nếu cần thứ tự + exactly-once).

- **Fan-out 1 message tới N hệ thống xử lý độc lập → chọn gì? Đáp án gọn:**
SNS → nhiều **SQS** (mỗi queue một bản, xử lý riêng).
- **SQS FIFO đảm bảo điều gì? Cần tham số nào? Đáp án gọn:** giữ đúng thứ tự + exactly-once processing; cần **MessageGroupId** (thứ tự) và **MessageDeduplicationId** (khử trùng, window 5 phút).
- **Message vào DLQ khi nào? Đáp án gọn:** khi số lần receive vượt **maxReceiveCount** mà chưa bị xóa (poison message).
- **Firehose khác Kinesis Data Streams ở điểm cốt lõi nào? Đáp án gọn:**
Amazon Data Firehose near-real-time, KHÔNG replay, tự nạp vào **S3/Redshift/OpenSearch/Splunk/Snowflake**; **KDS** real-time, replay được, phải tự viết consumer.
- ★ **CHECKPOINT Domain 1:** đã đạt **≥70%** ở MINI-MOCK Domain 1 (~25 câu) chưa? Nếu chưa → **KHÔNG** sang Tuần 6, ôn lại câu sai trước.



Tài nguyên tuần này

📁 **Đã crawl sẵn tài liệu AWS vào [resources/](#) — đọc offline được.**

- AWS Docs: **Amazon SQS** Developer Guide — Standard vs FIFO, visibility timeout, DLQ, long polling, DLQ redrive.
- AWS Docs: **Amazon SNS** Developer Guide — fan-out, message filtering (MessageBody & MessageAttributes), FIFO topics.
- AWS Docs: **Amazon Kinesis Data Streams** Developer Guide — shard, ordering, enhanced fan-out; **Amazon Data Firehose** Developer Guide.
- AWS Docs: **AWS Step Functions** Developer Guide — Amazon States Language, Standard vs Express, error handling (**Retry/Catch**), Distributed Map.
- AWS Docs: **Amazon ElastiCache** — Valkey / Redis vs Memcached, caching strategies (lazy loading, write-through, TTL).
- AWS Docs: **Amazon RDS Proxy** User Guide — connection pooling, VPC networking, reader endpoints, **Secrets Manager** / **IAM** auth.
- FAQ: **Amazon SQS** FAQs, **Amazon Kinesis** FAQs.
- Khoá học: Stephane Maarek — mục **SQS/SNS/Kinesis** messaging + **Step Functions**; Adrian Cantrill — Application services & caching.



Checklist hoàn thành Tuần 5

- ☐ Hoàn thành 4 buổi A/B/C/D

- ☐ Thuộc lòng bảng "PHẢI NHỚ" (số liệu **SQS/Kinesis/Step Functions**)
- ☐ Phân biệt được **SQS** vs **SNS** vs **Kinesis** qua keyword
- ☐ Hoàn thành 4 lab (fan-out, DLQ, state machine, bảng so sánh)
- ☐ **Đạt $\geq 70\%$ MINI-MOCK Domain 1 (~25 câu) — CHECKPOINT**
- ☐ Vượt Cổng tự kiểm tra